Test Execution Report: OpenCart Project

1. Executive Summary

This report summarizes the results of the comprehensive testing phase for the OpenCart project, covering Manual UI, API Automation, and Database validation cycles. The overall pass rate of **93.4%** demonstrates strong stability in core "happy-path" functionality.

⚠️ **Critical Finding:** Significant gaps were identified in backend validation and database constraints, particularly during **Negative Testing**—highlighting critical areas for immediate pre-production attention.

2. Test Execution Overview

The testing cycle covered 304 test cases across three distinct areas. The overall outcome indicates a stable application but exposes high-risk areas.

Test Type	Total Cases	Passed	Failed	Pass Rate	Status
Manual UI	108	106	2	**98.1%**	✅ Excellent
API Automation	156	148	8	**94.8%**	⚠️ Good
Database	40	30	10	**75.0%**	❌ Critical
TOTAL	**304**	**284**	**20**	**93.4%**	🟡 Stable

3. Detailed Analysis of Test Cycles

3.1. Manual UI Testing (Pass Rate: 98.1%)

Scope: Verified core user flows including Footer Links, Checkout, Search, Homepage, and Cart functionality.
Result: The user interface is highly stable for typical user interactions.
Key Failures (2 Failures):

- **TC_SR_06**: Search in Product Descriptions failed to return expected results (Functional

defect).

- **TC_CO_02**: Checkout validation issue, likely related to improper handling of empty postcode fields (Validation defect).

3.2. API Automation (Pass Rate: 94.8%)

Scope: Validated essential REST endpoints for Products, Brands, Search, Login, and Account Management.

8 Failures Summary: The primary issue is the **incorrect HTTP Status Code mapping**, particularly for negative or non-creation scenarios.

Test Case	Description	Expected Status	Actual Status	Issue
API 11, 12	Create/Delete Account	201 (Created)	200 (OK)	Incorrect response code for resource state change.
API 8_N, 10_N	Login (Missing/Invalid Creds)	400 (Bad Request)	200 (OK)	**High Risk:** Server masked error; security/validation failure.
API 5	Search Product	200	200	Functional failure: Correct status, but product results were incorrect/incomplete.
API 7, 13, 14	Account Management	-	-	Functional failure (unexpected login failure, message mismatch, details check failed).

3.3. Database Testing (Pass Rate: 75.0%)

Scope: Verified data integrity and constraints for Customers, Products, Orders, and Categories tables. This cycle showed the lowest pass rate and the highest risk.

Critical Failures (10 Failures):

The failures (DB_31 to DB_40) are primarily due to the **lack of robust backend and database-level constraint handling**. The database accepted records that should have been blocked.

Examples of Missing Constraints:

- NOT NULL: Null values accepted for mandatory fields (e.g., Customer Email).
- CHECK: Negative or illogical values accepted (e.g., Negative Price for a Product, Quantity = 0 for Order).
- FOREIGN KEY: Invalid Foreign Keys were not properly rejected, leading to orphaned data.

4. Defect and Risk Distribution

The identified defects are clustered around backend validation logic and are categorized by severity.

Severity	Defect Type	Risk Assessment	Recommended Action
HIGH	Missing Database Constraints	Data corruption & Security vulnerability.	**IMMEDIATE**: Implement NOT NULL, CHECK, and FOREIGN KEY constraints.
MEDIUM	API Status Code Mismatches	Poor integration & troubleshooting difficulty.	**PRIORITY**: Standardize API error responses (400, 401, etc.).
LOW	Minor UI Glitches	Minor user experience friction.	Resolve specific UI test failures (Search, Postcode validation).

5. Conclusion & Recommendations

The OpenCart application has a **good foundation**, as evidenced by the high pass rates in Manual UI and Positive API testing.

System Vulnerability: The critical finding is that the system is currently highly vulnerable to bad data input, which could lead to severe application issues in production.

Key Recommendations (Prioritized):

1. **Backend Integrity (HIGH):** Enforce all critical database constraints (Section 3.3).
2. **API Contract (MEDIUM):** Correct API response codes for error and state-change scenarios (Section 3.2).

Failure to address these backend issues could lead to irreversible data integrity problems and critical application failures.